Dokumentace projektu

SÚKL API

REST API pro agregaci dat z Českého registru léčivých přípravků

Vsevolod Pokhvalenko
Lukáš Csáder
Michael Janeček
Michal Žatečka

19. dubna 2026

Obsah

1 Úvod a cíl projektu **3**

2 Autoři a rozdělení práce **3**

3 Architektura a technologické volby **4**
3.1 Přehled architektury 4
3.2 Volba technologií 4

4 Datové zdroje **5**
4.1 Přehled datových sad 5
4.2 Načítání dat 6

5 Databázové schéma **6**

6 Popis API endpointů **7**

6.1 Přehled skupin 7
6.2 Společné vlastnosti 8

7 Použité knihovny 8

8 Testování 9
8.1 E2E (end-to-end) testy 9
8.2 Unit testy 10
8.3 Spuštění testů 10

9 Instalace a spuštění 10
9.1 Požadavky 10
9.2 Postup 10
9.3 Proměnné prostředí 10

10 Testovací webová aplikace (sukl-FE) 11
10.1 Technologie 11
10.2 Stránky a pokrytí API 11
10.3 Architektura 12
10.4 Spuštění 12

11 Omezení a možná rozšíření 12
11.1 Současná omezení 12
11.2 Možná budoucí rozšíření 12

12 Závěr 13

1 Úvod a cíl projektu

Cílem projektu je implementace REST API, které agreguje a zpřístupňuje veřejná data Státního ústavu pro kontrolu léčiv (SÚKL). SÚKL zveřejňuje rozsáhlé datové sady ve formátu CSV popisující léčivé přípravky registrované v České republice, jejich složení, ceny, výpadky v dodávkách, preskripci, lékárny a další. Tato data jsou volně dostupná na portálu otevřených dat SÚKL (https://opendata.sukl.cz), avšak jejich přímé využití vyžaduje stažení velkých CSV souborů a jejich lokální zpracování.

Výsledné API umožňuje programatický přístup k těmto datům prostřednictvím standardního HTTP rozhraní s JSON odpověďmi, stránkováním, filtrováním a dokumentací ve formátu OpenAPI (Swagger). API je navrženo jako **read-only** (pouze čtení) – všechny endpointy používají metodu GET.

2 Autoři a rozdělení práce

Projekt realizoval tým čtyř studentů. Níže je uveden přehled příspěvků každého člena na základě obou repozitářů (`sukl-api` a `sukl-FE`).

Autor	Příspěvky
Vsevolod Pokhvalenko (xpokhv00)	Iniciální nastavení projektu a základní architektura Express aplikace. Implementace endpointů: léčiva, účinné látky, lékárny, preskripce, ATC browser, změny registrací. Validační middleware (Zod), rate limiting, Swagger/OpenAPI dokumentace. Typová bezpečnost (rozšíření Express typů, `z.infer<>` na controllerech). E2E testovací suite (Jest + Supertest), changelog, projektová dokumentace. Frontend: navigační lišta, sekce preskripčních statistik na detailu léčiva.
Michael Janeček (YanehCheck)	Endpoint pro hodnocení rizika výpadků (`/statistics/supply-risk`) s TTL cache. Endpoint organizací (`/organizations`) včetně čtyř sub-endpointů (léčiva, výpadky). Všechny tři varianty endpointů výpadků (`/disruptions`). Metadatové endpointy (`/meta`), filtrování preskripce dle ATC kódu, celkový součet preskripce. Vyhledávání účinných látek se synonym matching a in-memory řazením dle relevance. CI/CD pipeline (GitHub Actions), automatické spouštění datových loaderů. Oprava normalizace SÚKL kódu, infrastruktura E2E testů.

Autor	Příspěvky
Michal Žatečka (MikiiN)	Validace cizích klíčů při načítání dat (ochrana před nevalidními FK záznamy). Opravy kódování zdrojových souborů a dávkového zpracování. Oprava sloupce zdrojového statusu registrace v loaderu. Frontend (sukl-FE): inicializace projektu (Vite + React), základní seznam léčiv s vyhledáváním a stránkováním, detail léčiva, extrakce komponenty `MedicationCard`, oprava klikatelných karet.
Lukáš Csáder (LukasCsader)	Generická továrna `createLoader` sdílená všemi datovými loadery. Loadery pro hlavní tabulky: léčiva, účinné látky, organizace, lékárny, cenové reporty. Hlavní soubor `seed.ts` (vstupní bod pro naplnění databáze). Oprava FK pro otevírací dobu lékáren.

3 Architektura a technologické volby

3.1 Přehled architektury

Aplikace je postavena na klasické třívrstvé architektuře:

1. **Vrstva routování a validace** – Express router + Zod schémata
2. **Vrstva řadičů a služeb** – Controllers delegují na Services
3. **Datová vrstva** – Prisma ORM komunikuje s PostgreSQL databází

Listing 1: Zjednodušený tok požadavku

```
HTTP Request
  -> Express Router
    -> Zod Validation Middleware
      -> Controller
        -> Service
          -> Prisma ORM
            -> PostgreSQL (Supabase)
```

3.2 Volba technologií

Node.js s TypeScriptem byl zvolen jako runtime prostředí. TypeScript přináší statickou typovou kontrolu, která je klíčová při práci s komplexními datovými modely (24 Prisma modelů, stovky polí). Chyby při mapování dat z CSV jsou odhaleny v době kompilace, nikoliv až za běhu.

Express byl zvolen oproti nativnímu Node.js HTTP modulu kvůli jeho middleware-based architektuře a abstrakci nad routováním. Native HTTP server v Node.js vyžaduje

manuální implementaci routing logiky, parsování request body a centralizaci error handlingu, což zvyšuje komplexitu při větším počtu endpointů. Express tyto mechanismy standardizuje a umožňuje modulární strukturu aplikace, která lépe odpovídá vrstvené architektuře projektu (router → controller → service → data layer).

Prisma ORM s adaptérem `@prisma/adapter-pg` byl zvolen pro správu databázového schématu a dotazů. Prisma poskytuje migrace s verzováním (`prisma migrate`), typově bezpečné dotazy generované ze schématu a přehlednou definici relací v souboru `schema.prisma`. Adaptér `adapter-pg` umožňuje přímé využití connection poolingu přes `pg` bez zprostředkující vrstvy.

PostgreSQL na Supabase byl zvolen jako databázové řešení. Supabase poskytuje spravovanou PostgreSQL instanci s connection poolingem přes PgBouncer, SSL připojením a bezplatným plánem dostačujícím pro tento projekt.

Zod byl zvolen pro validaci vstupních parametrů. Zod umožňuje definovat schémata dotazovacích parametrů (query strings) přímo v TypeScriptu, automaticky provádí typovou koerci (string → number pro parametr `page`) a generuje srozumitelné chybové zprávy ve formátu JSON. Schémata jsou sdílena přes `src/schemas/` a znovupoužívána ve validačním middlewaru.

Pino přes `pino-http` zajišťuje strukturované logování HTTP požadavků ve formátu JSON s minimálním výkonnostním dopadem.

Swagger/OpenAPI dokumentace je generována automaticky ze JSDoc komentářů v souborech routerů pomocí `swagger-jsdoc` a servírována přes `swagger-ui-express` na cestě `/docs`.

4 Datové zdroje

Veškerá data pocházejí z portálu otevřených dat SÚKL (https://opendata.sukl.cz). Data jsou stahována automaticky pomocí modulu `src/loaders/downloadAndLoad.ts` a načítána do databáze přes upsert operace (vložení nebo aktualizace dle primárního klíče).

4.1 Přehled datových sad

Datová sada	Obsah
DLP	Databáze léčivých přípravků – hlavní datová sada. Obsahuje přípravky, složení, ATC klasifikaci, organizace, lékové formy, cesty podání, kategorie výdeje, statusy registrace a další číselníky.
MR	Hlášení o výpadcích v dodávkách léčivých přípravků.

Datová sada	Obsah
REG	Nové a zrušené registrace léčivých přípravků (včetně zrušení v rámci EU).
LEKARNY	Seznam lékáren v ČR s adresami a provozní dobou.
LEK-13	Měsíční reporty maximálních cen a výše úhrady přípravků.
DIS-13	Historická data výpadků v dodávkách.
ERECEPT	Historie předepsaných léčivých přípravků ze systému eRecept – statistiky preskripce dle okresu.
POVINNOST_DODAVEK	Povinná množství přípravků zajišťovaná držitelem rozhodnutí o registraci.

4.2 Načítání dat

Načítání probíhá ve dvou krocích:

1. **Stažení** – ZIP archivy nebo přímé CSV soubory jsou staženy z SÚKL serverů a uloženy do dočasného adresáře.
2. **Zpracování** – Soubory jsou streamově parsovány pomocí `fast-csv` a `iconv-lite` (SÚKL exportuje data v kódování Windows-1250). Záznamy jsou dávkově vkládány do databáze přes `prisma.upsert`.

Načítání lze spustit manuálně nebo je plánováno pomocí `node-cron`.

5 Databázové schéma

Schéma je definováno v souboru `prisma/schema.prisma` a obsahuje **24 modelů** rozdělených do tematických skupin:

- **Jádro přípravků:** `Medication`, `Substance`, `Composition`, `Synonym`, `MedicationDocument`, `MedicationDoping`
- **Klasifikační číselníky:** `AtcNode`, `PharmaceuticalForm`, `AdministrationRoute`, `DispensingCategory`, `RegistrationStatus`, `DependencyCategory`, `MedicationType`, `IndicationGroup`, `NarcoticCategory`, `DopingCategory`
- **Organizace a distribuce:** `Organization`, `Pharmacy`, `PharmacyHour`
- **Transakční data:** `Prescription`, `PriceReport`, `Disruption`, `DispensingRestriction`, `RegistrationChange`

Migrace jsou verzovány v adresáři `prisma/migrations/` a jsou nasazovány příkazem `npx prisma migrate deploy`.

6 Popis API endpointů

Kompletní interaktivní dokumentace všech endpointů včetně příkladů požadavků a odpovědí je dostupná přes Swagger UI na adrese `/docs` po spuštění serveru. Níže je uveden přehled skupin endpointů.

6.1 Přehled skupin

Prefix	Metody	Popis
`/medications`	GET	Seznam léčivých přípravků s filtrováním dle názvu, ATC kódu, účinné látky, formy a kategorie výdeje. Detail přípravku včetně složení, dokumentů, výpadků a cenových reportů. Statistiky preskripce dle přípravku.
`/substances`	GET	Vyhledávání účinných látek dle názvu nebo INN (včetně synonym). Detail látky s výčtem přípravků, které ji obsahují.
`/pharmacies`	GET	Seznam lékáren s filtrováním dle města, PSČ, zásilkového výdeje a pohotovostního provozu. Detail lékárny s otevírací dobou.
`/disruptions`	GET	Výpadky v dodávkách – všechny, pouze aktivní, aktivní s náhradními přípravky. Historie výpadků pro konkrétní přípravek a jeho doporučené náhrady.
`/atc`	GET	Procházení ATC klasifikační hierarchie. Detail ATC uzlu s počtem registrovaných přípravků.
`/prescriptions`	GET	Statistiky preskripce dle okresu, roku a měsíce. Nejprescribovanější přípravky. Celkový součet předepsaných balení dle filtru.
`/organizations`	GET	Držitelé rozhodnutí o registraci s filtrováním dle názvu a země. Léčiva a výpadky organizace.
`/registration-changes`	GET	Nové, zrušené a EU-zrušené registrace s filtrováním dle typu, názvu a držitele.
`/statistics`	GET	Dashboard rizika výpadků dle ATC skupin – kombinuje počet aktivních výpadků, objem preskripce a chybějící množství do skóre rizika.

Prefix	Metody	Popis
`/meta`	GET	Referenční číselníky: lékové formy, cesty podání, kategorie výdeje, statusy registrace, dopingové kategorie, typy léčiv, indikační skupiny, narkotické kategorie.
`/health`	GET	Kontrola stavu serveru.

6.2 Společné vlastnosti

Všechny listovací endpointy podporují stránkování přes parametry `page` (výchozí: 1) a `limit` (výchozí: 20, maximum: 100). Odpověď vždy obsahuje objekt `meta` s celkovým počtem záznamů a počtem stránek.

Textové filtry jsou case-insensitive. Filtrování dle kódů (ATC, SÚKL kód) funguje jako prefix match.

API je chráněno rate limitingem: **120 požadavků za 60 sekund** na IP adresu.

7 Použité knihovny

Knihovna	Verze	Účel
express	5.2.1	HTTP server framework s nativní podporou async chyb
@prisma/client	7.6.0	Typově bezpečný ORM klient generovaný ze schématu
@prisma/adapter-pg	7.6.0	PostgreSQL driver adapter pro Prisma s connection poolingem
prisma	7.6.0	CLI nástroj pro správu migrací a generování klienta
pg	8.20.0	Node.js PostgreSQL klient (využíván Prisma adaptérem)
zod	4.3.6	Validace a typová koercie vstupních parametrů
dotenv	17.3.1	Načítání konfigurace z `.env` souboru
helmet	8.1.0	Bezpečnostní HTTP hlavičky
cors	2.8.6	Cross-Origin Resource Sharing
express-rate-limit	8.3.2	Ochrana API rate limitingem
pino	10.3.1	Výkonné strukturované logování
pino-http	11.0.0	HTTP request/response logging middleware
swagger-jsdoc	6.2.8	Generování OpenAPI specifikace z JSDoc komentářů
swagger-ui-express	5.0.1	Servírování Swagger UI
fast-csv	5.0.5	Streamové parsování CSV souborů

Knihovna	Verze	Účel
iconv-lite	0.7.2	Konverze kódování (Windows-1250 → UTF-8)
unzipper	0.12.3	Rozbalování ZIP archivů při stahování dat
node-cron	4.2.1	Plánování automatického stahování dat
tsx	4.21.0	TypeScript runner pro vývoj (bez kompilačního kroku)
jest	30.3.0	Testovací framework
supertest	7.2.2	HTTP integrace pro testování Express endpointů
ts-jest	29.4.9	TypeScript transformátor pro Jest
typescript	6.0.2	Statická typová kontrola

8 Testování

Projekt obsahuje automatizované testy v adresáři `src/__tests__/` rozdělené na:

8.1 E2E (end-to-end) testy

Testy spouští skutečný Express server a odesílají HTTP požadavky přes `supertest`. Každá skupina endpointů má vlastní testovací soubor:

- `medications.test.ts` – filtrování, stránkování, detail přípravku, 404 pro neexistující kód
- `substances.test.ts` – vyhledávání dle názvu a INN, synonyma
- `pharmacies.test.ts` – filtrování dle města, zásilkový výdej, otevírací doba
- `disruptions.test.ts` – aktivní výpadky, výpadky s náhradami, filtrování dle ATC
- `atc.test.ts` – procházení ATC stromu, detail uzlu
- `prescriptions.test.ts` – statistiky, top přípravky, celkový součet
- `organizations.test.ts` – vyhledávání, léčiva a výpadky organizace
- `registrationChanges.test.ts` – filtrování dle typu změny
- `statistics.test.ts` – supply risk dashboard
- `meta.test.ts` – všechny číselníkové endpointy
- `root.test.ts` – health check, kořenový endpoint

8.2 Unit testy

- `dataDownloader.test.ts` – testování stahování a rozbalování datových souborů
- `dataLoaderScheduler.service.test.ts` – testování plánování automatického načítání dat

8.3 Spuštění testů

```
npm test
```

9 Instalace a spuštění

9.1 Požadavky

- Node.js v22 LTS nebo novější
- Přístup k PostgreSQL databázi (např. Supabase)

9.2 Postup

```
# 1. Instalace závislostí
npm install

# 2. Konfigurace prostředí
cp .env.example .env
# Vyplnit DATABASE_URL a DIRECT_URL v souboru .env

# 3. Nasazení databázových migrací
npx prisma migrate deploy

# 4. Generování Prisma klienta
npx prisma generate

# 5. Spuštění vývojového serveru
npm run dev
```

Server poté běží na `http://localhost:3000` a Swagger dokumentace je dostupná na `http://localhost:3000/docs`.

9.3 Proměnné prostředí

Listing 2: .env.example

```
DATABASE_URL=postgresql://user:password@host:5432/dbname
```

```
DIRECT_URL=postgresql://user:password@host:5432/dbname
PORT=3000
NODE_ENV=development
```

10 Testovací webová aplikace (sukl-FE)

Součástí projektu je testovací React aplikace `sukl-FE`, která slouží jako demonstrace a ověření funkčnosti API v reálném uživatelském rozhraní. Aplikace je záměrně minimalistická – jejím cílem není produkční UI, ale pokrytí všech endpointů API a ověření správnosti dat.

10.1 Technologie

- **React 19** s **TypeScriptem** – komponentový UI framework s typovou bezpečností
- **React Router DOM v7** – klientské routování s deep-linkingem na detail stránky
- **Vite** – build nástroj a vývojový server s hot-reload
- Žádná externí UI knihovna – vlastní CSS třídy

10.2 Stránky a pokrytí API

Aplikace obsahuje 13 stránek mapovaných na skupiny endpointů API:

Stránka (route)	Popis
`/`	Seznam léčivých přípravků s vyhledáváním (debounce 400 ms) a stránkováním
`/:suklCode`	Detail přípravku – složení, ceny, výpadky, preskripční statistiky
`/substances`	Vyhledávání účinných látek
`/substances/:id`	Detail látky se složením přípravků
`/pharmacies`	Seznam lékáren s filtry (město, zásilkový výdej, pohotovost)
`/pharmacies/:id`	Detail lékárny s otevírací dobou
`/disruptions`	Výpadky ve třech záložkách: všechny / aktivní / aktivní s náhradami
`/atc`	Interaktivní stromové procházení ATC klasifikace
`/prescriptions`	Statistiky preskripce – nejprescribovanější přípravky a filtrovaný přehled
`/organizations`	Seznam organizací (držitelé registrací) s filtrováním

Stránka (route)	Popis
`/organizations/:code`	Detail organizace s léčivy a výpadky
`/registration-changes`	Log změn registrací s filtrováním dle typu
`/statistics`	Dashboard rizika výpadků dle ATC skupin

10.3 Architektura

Všechna volání API jsou centralizována v souboru `src/services/api.ts` přes generickou funkci `apiFetch<T>`, která obaluje `fetch` API s typovými parametry a chybovým handlingem. Stav aplikace je řízen lokálními React hooky (`useState`, `useEffect`) bez externí state management knihovny. Znovupoužitelné komponenty (`Pagination`, `SearchBar`, `MedicationCard`) jsou sdíleny napříč stránkami.

10.4 Spuštění

```
cd ../sukl-FE
npm install
npm run dev
# Aplikace běží na http://localhost:5173
# Vyžaduje spuštěné API na http://localhost:3000
```

11 Omezení a možná rozšíření

11.1 Současná omezení

- **Pole `level` a `parentCode` v ATC** – Zdrojová data SÚKL neobsahují explicitní úroveň ani rodičovský kód v ATC klasifikaci. Hierarchie je odvozitelná z délky kódu (1, 3, 4, 5, 7 znaků), ale pole jsou v databázi prázdná.
- **Pouze čtení** – API neposkytuje žádné write operace (POST, PUT, DELETE). Data jsou aktualizována výhradně přes datové loadery.
- **Žádná autentizace** – API je veřejné, ochrana je zajištěna pouze rate limitingem. Pro produkční nasazení by bylo vhodné doplnit API klíče.

11.2 Možná budoucí rozšíření

- Automatické periodické stahování a aktualizace dat ze SÚKL
- Fulltextové vyhledávání přes PostgreSQL `tsvector`
- Cachování výsledků statistických dotazů (Redis)
- GraphQL vrstva nad existujícími službami

12 Závěr

Výsledkem projektu je funkční REST API zpřístupňující data Státního ústavu pro kontrolu léčiv. API pokrývá všechny hlavní datové sady dostupné na portálu SÚKL a poskytuje 41 endpointů ve 10 skupinách s filtrováním, stránkováním a kompletní OpenAPI dokumentací. Projekt demonstruje použití moderního Node.js stacku s důrazem na typovou bezpečnost (TypeScript + Prisma + Zod), automatizované testování (Jest + Supertest) a standardizovanou dokumentaci (Swagger/OpenAPI).